

Verify your knowledge

- Do I have to configure the `pg_hba.conf` file before starting physical replication?

Yes, you do.

See the section *Managing streaming replication* for more details.

- Is it possible on PostgreSQL to make an asynchronous replication?

Yes, it is possible; it's the default configuration.

See the section *Managing streaming replication* for more details.

- Is it possible on PostgreSQL to make a synchronous replication?

Yes, it is possible, by modifying the `postgresql.conf` on the primary server and the `postgresql.auto.conf` on the replica server.

See the section *Synchronous replication* for more details.

- Is it possible on PostgreSQL to make a cascading replication?

Yes, it is possible, by using a `pg_basebackup` command that takes data from the replica server.

See the section *Cascading replication* for more details.

- Is it possible to promote a replica node to a primary node?

Yes, it is, by using the `pg_ctl promote` command.

See the section *Promoting a replica server to a primary* for more details.

References

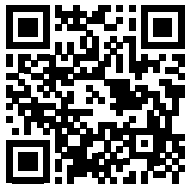
- Wal level settings official documentation: <https://www.postgresql.org/docs/current/runtime-config-wal.html#GUC-WAL-LEVEL>
- Pg_basebackup command official documentation: <https://www.postgresql.org/docs/current/app-pgbasebackup.html>
- Replica monitoring official documentation: <https://www.postgresql.org/docs/current/monitoring-stats.html#PG-STAT-REPLICATION-VIEW>

- Replica configuration official documentation: <https://www.postgresql.org/docs/current/runtime-config-replication.html>
- High Availability, Load Balancing, and Replication official documentation: <https://www.postgresql.org/docs/current/high-availability.html>

Learn more on Discord

To join the Discord community for this book – where you can share feedback, ask questions to the author, and learn about new releases – follow the QR code below:

<https://discord.gg/jYWCjF6Tku>



18

Logical Replication

In the previous chapter, we talked about WAL segments and physical replication in synchronous, asynchronous, and cascading modes. In this chapter, we will cover the topic of logical replication. We will look at how to perform a logical replica, how a logical replication is different from a physical replication, and when it's better to use logical replication instead of physical. We'll also see that logical replication can be used to make a PostgreSQL hot upgrade. This chapter is intended as an introduction to logical replication; for further information, refer to more advanced texts, such as *Mastering PostgreSQL*, by Hans-Jürgen Schönig.

This chapter covers the following topics:

- Understanding the basic concepts of logical replication
- Comparing logical replication and physical replication
- Exploring a logical replication setup and new logical replication features on PostgreSQL 16

Technical requirements

For this chapter, you will find three Docker environments in the repository:

- `chapter18_logical_clear`: contains two PostgreSQL installations ready for the configuration of a new logical replication.
- `chapter18_logical_ready`: contains two PostgreSQL installations with a new logical replication already active.
- `chapter18_physical_logical`: contains three PostgreSQL installations with a new logical replication made using a physical replication.

If you want to understand how to configure a new logical replication, you should use the first Docker environment; however, if you want to skip all the topics about configuration, you can use the second one. In the second Docker environment, which you can find on the publication server, you will find the `forumdb` database that we've used so far in the book; you'll also find a logical replication of *just* the `users` table. Finally, the `chapter18_physical_logical` Docker environment will be used for the last section of this chapter.

Understanding the basic concepts of logical replication

Logical replication is a method that we can use to replicate data based on the concept of identity replication. `REPLICA IDENTITY` is a parameter present in table management commands (such as `CREATE TABLE` and `ALTER TABLE`); this parameter is used by PostgreSQL to obtain additional information within WAL segments, to recognize which tuples have been eliminated and which tuples have been updated. The `REPLICA IDENTITY` parameter can take four values:

- `DEFAULT`
- `USING INDEX index_name`
- `FULL`
- `NOTHING`

The concept behind logical replication is to pass the logic of the commands executed on the primary machine to the server and not the exact copy of the blocks to be replicated, byte by byte. At the heart of logical replication, there is a reverse engineering process that, starting from the WAL segments and using a logical decoding process, is able to extrapolate the original SQL commands and pass them on to the replication machine, using a logical decoding process.

Let's see a flow chart that shows how PostgreSQL internally executes queries:

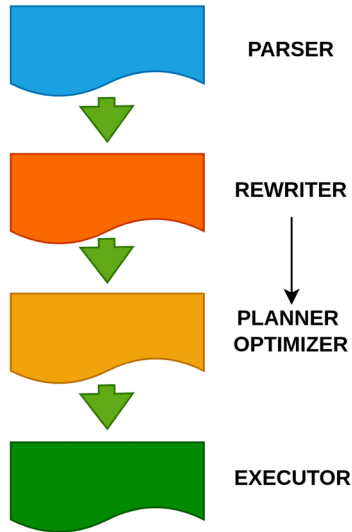


Figure 18.1: An illustration of the backend process

As we can see, a query, before being executed, requires several internal steps; this is because the system tries to execute the query in the best possible way, according to the conditions prevailing at that moment in the database. Now, suppose we want to replicate the data logically; at this point, we have two possibilities in front of us:

- We can capture commands before they get to the parser and transfer these commands to a second machine.
- We can try, in some way, to get the queries that are already parsed.

The first method is implemented by systems designed prior to native logical replication, which was based on triggers; an example of the application of this method can be found on Slony (<https://www.slony.info/>).

The second method is used in logical replication.

In logical replication, we will take the commands to be sent to the replica server within the WAL segments. The problem is that within the WAL segments, we have a physical representation of the data. In other words, within the WAL segments, the data is ready to be sent or archived to make physical copies, not logical copies.

Logical replication is based on the concept that WAL segments, after being processed through a logical decoding process that reverses the physical information in a logical information, are made available through a publication mechanism. The primary will then start a publication process, and the replica will start a subscription process that, by connecting to the primary's publication, is able to pass the decoded instructions directly to the query executor of the replica machine.

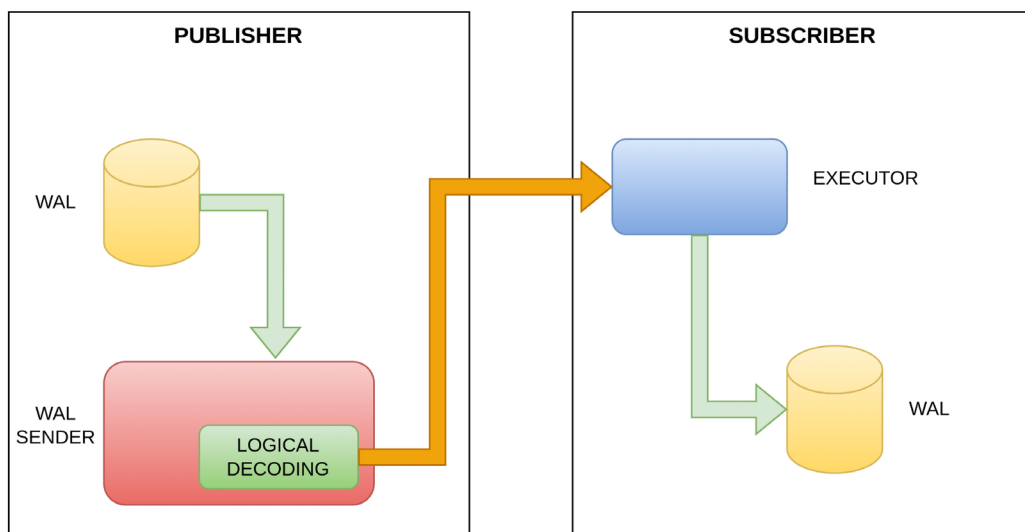


Figure 18.2: A logical replication schema

As we can see from the diagram, using a reverse engineering process, instructions are retrieved from the WAL segments, and these instructions are ready to be processed by the executor of the replica server without any parsing action. This second method is much faster than the first method. The first method was the only one available for PostgreSQL versions prior to 9.4; starting from 9.4, there is an extension called `pglogical`, and since version 10.0, the logical replica has become native.

Comparing logical replication and physical replication

Let's now examine how a logical replica differs from a physical replica:

- One of the positive characteristics of physical replicas is their speed. However, a distinct disadvantage is that we have to replicate all the databases in the cluster. Using a physical replica, it is not possible to replicate a single database belonging to an instance of PostgreSQL, and it is not possible to replicate only some tables of a database. Logical replication is a little bit slower than physical replication, but by using logical replication, we can decide which databases we want to replicate within a cluster and/or which tables we want to replicate within a single database.
- Physical replication is only possible if the two servers have the same version of PostgreSQL. With logical replication, since the logical instruction to be executed is passed to the replica server, it is also possible to perform replications between different versions of PostgreSQL.
- In a physical replication, with the exception of operations on temporary and unlogged tables, all operations are replicated. In a logical replication, only **data manipulation language (DML)** operations are replicated, and **data definition language (DDL)** operations such as ALTER and TABLE operations are not replicated.
- Physical replication creates, by definition, a physical copy; it binarily replicates all the contents of the primary server that pass through the WAL onto the replica. Logical replication, on the other hand, only replicates the instructions, that is, the statements that we give to the replica server.
- Physical replication, with the exception of unlogged tables, makes an identical copy of the primary on the replica server. Physical replication copies absolutely everything; thus, because the copy is physical at the page level, we copy not just the data but also any bloat associated with it. Sometimes, this can be useful, for example, if we want to simulate the exact behavior of the production server in our test environment.
- Logical replication, however, through a reverse engineering mechanism, passes the queries to be executed directly to the query executor of the replica machine. For example, if we want to get a copy of our database to start with a low bloating percentage, we can perform a logical replica on a second machine, and the second machine will begin from a very clean starting point. This is because all data will be passed in a non-physical, but logical, way to the second server. Additionally, it is possible to replicate data between different versions of PostgreSQL servers this way.



Note that, because we can do replications between different versions of PostgreSQL, logical replication is a tool that can be used to perform PostgreSQL hot upgrades.

Exploring a logical replication setup and new logical replication features on PostgreSQL 16

Let's now explore how to perform logical replication. In this section, we will prepare the environment we need to be able to perform our logical replication.

Logical replication environment settings

Suppose we have two machines, which we will call `pg_pub` and `pg_sub`. We must remember to set our internal DNS, or the `/etc/hosts` file, so that `pg_pub` can reach `pg_sub`; for example, for the `pg_pub` server, the primary server will have an IP of `192.168.144.3`, and for the `pg_sub` server, the replica server will have an IP of `192.168.144.2`. If you use the `chapter18` container, you can execute:

```
chapter_18$ bash run-pg-docker.sh chapter18_logical_clear
```

Once you are inside the first container, you can open another bash terminal and execute:

```
chapter_18$ bash run-pg-docker_replica.sh chapter18_logical_clear
```

Now, let's check whether there is a connection between the two servers:

```
postgres@pg_pub:~$ ping pg_sub
PING pg_sub (192.168.144.3) 56(84) bytes of data.
64 bytes from chapter18_logical_clear_learn_postgresql_sub_1.chapter18_
logical_clear_default (192.168.144.3): icmp_seq=1 ttl=64 time=0.094 ms

postgres@pg_sub:~$ ping pg_pub
PING pg_pub (192.168.144.2) 56(84) bytes of data.
64 bytes from chapter18_logical_clear_learn_postgresql_pub_1.chapter18_
logical_clear_default (192.168.144.2): icmp_seq=1 ttl=64 time=0.070 ms
```

As shown here, there is a connection between the two servers.

The replica role

In order to perform a logical replication, as we already did in the previous chapter when we talked about physical replication, we need a database user with replication permissions. So, let's create the following user on the publication server:

```
postgres=# CREATE USER replicarole WITH REPLICATION ENCRYPTED PASSWORD
'LearnPostgreSQL';
CREATE ROLE
```

This user will be used to manage logical replication.

Primary server — postgresql.conf

Now, we will modify the `postgresql.conf` file on both servers; this is to ensure that the two servers listen on port 5432 for network interfaces. We will then modify some other values to try to optimize the logical replication procedure:

1. First, we add the following line to the end of the `postgresql.conf` file on the publication server:

```
# Add settings for extensions here
listen_addresses = '*'
wal_level = logical
max_wal_senders = 10
```

Now, let's look at each parameter in turn:

- `listen_addresses = '*':` This way, we make PostgreSQL listen on port 5432 on all network interfaces. We could also simply add the IP address of the interface where we want the PostgreSQL service to listen.
- `wal_level = logical:` We changed the value from `replica` (default) to `logical`; this way, PostgreSQL, in addition to all the information present in the `wal_level = replica` model, will add more information so that it can make the reverse engineering process possible. With `wal_level = logical`, we make logical replication possible.
- `max_replication_slots = 10:` This value must be set to at least one for each subscriber, plus those necessary for the initialization of the tables.
- `max_wal_senders = 10:` This value must be set to a number at least equal to one for each replication slot, plus those necessary for physical replication.